Ethan Patterson
Nicolas Cabrera

Hw4 q1

a.) This is an algorithm that tricks the longest increasing subsequence algorithm into working for this specific scenario. Longest increasing subsequence, by default, finds the longest sequence of numbers in an array that increases numerically. The concept of having “classes” almost in the style of a greedy interview problem(especially because it mentions it) is entirely a red herring. In our case, we’re looking for the longest sequence of classes that we can take without overlapping, including travel time. The core concept of longest increasing subsequence works as is, where we iterate over every option, then try to build the subsequence to reach it from the subsequences before it. The only change to that is we take in the travel time condition, ensuring that the next class after our current longest one is actually reachable before it starts. Otherwise, it *almost* works as is. The last piece is that we need to sort the input. With traditional LIS, we can have a series like 3 1 2 4 and the longest subsequence would be of length 3, consisting of 1, 2, and 4. In our case, ordering like that is a bit more important. If those numbers are our “start times”, we don't care that the ordering has 3 before 1. Chronologically, 1 still starts before 3, and the algorithm doesn’t work right unless we make it be the case. The equivalent case in a traditional version of this algo would be 4 9 5, where 5 is ignored because it is smaller than 9, but in our case we want to accept 4 5 9 as a reasonable ordering(assuming these are start times). As such, we’ll perform a quick insertion sort on the start times before touching the main algorithm, which ensures that our subsequence is “technically always increasing”(as in this pretend example it would now be 1 2 3 4), so as long as we don’t overlap with travel time, we’ll get the right answer.

b.)

```
Int findMinTravelTime(courseTimes, travelTimes):
        courseTimes = modifiedInsertionSort(courseTimes) // sort on start times
        subseqList[courseTimes.length]
        longestSubseq = 0
        For i=0; i<courseTimes.length; i++
                subseqList[i] = 1
                For j=0 j<i j++
                        RealTravelTime = travel time from j to i + finish time of j
                        If realTravelTime < start time of i and longest subseq of i < longest subseq
of j + 1:
                                subseqList[i] = subseqList[j] + 1
                        If subseqList[i] > longestSubseq: longestSubseq = subseqList[i]

        Return longestSubseq
```

c.) As a baseline foundation, the algorithm for longest increasing subsequences works. This is known and established. It is a dynamic programming solution that uses smaller subsequences to build larger ones. Adding our travel time condition to ensure the next class is takable without being late on travel, and an easy condition to add that won't compromise the integrity of the algorithm, as fundamentally, any case where it would accept those, *simply won't work*. The last change is adjusting the ordering. Ignoring the algorithm, changing the order that the classes are presented does not matter as the start time is a feature of each class independent of the ordering. In the case of our algorithm, we need to tell it that everything is technically increasing as long as it can fit in the schedule, and the algorithm will naturally do the rest to find the most classes possible. This cleans up cases where a class may be ignored because it is below the currently selected class start time wise and should reasonably go in, but does not because it is after it in the ordering.

d.) n^2

e.) Longest increasing subsequence at its core is n^2. This is because the other loop is iterating over n, and the inner loop is at most n, which is an n^2 runtime as everything inside them is constant time. We do not change this core much, the only nonconstant time complexity change is the initial insertion sort pass, which is also at worst n^2, and n^2 + n^2 time complexity wise is 2n^2 which is still just n^2 at the end of the day.